Module 10

Pandas for Data Manipulation

Python Data Science Training — 3 Hours

Topics Covered:

10.1 Introduction to Pandas
10.2 Series
10.3 DataFrames
10.4 Data Indexing and Selection
10.5 Data Import / Export ★ Core Topic
10.6 Data Cleaning
10.7 Data Transformation
10.8 Grouping and Aggregation
10.9 Merging and Joining
10.10 Time Series
10.11 Advanced Topics
10.12 Lab: End-to-End Data Wrangling

This document serves as both Trainer Slides and Post-Training Study Material

10.1 Introduction to Pandas

Pandas is an open-source data manipulation and analysis library for Python. It provides fast, flexible, and expressive data structures designed to make working with structured (tabular) and time-series data easy and intuitive.

Why Pandas?

- Handle missing data gracefully
- Easy data alignment and merging
- Group-by operations for split-apply-combine workflows
- Powerful IO tools — read from CSV, Excel, SQL, JSON, HTML, Parquet, and more
- Label-based and integer-based slicing/indexing
- Integrates tightly with NumPy, Matplotlib, Scikit-learn

Real-World Use Cases

Domain	Example Use Case
Finance	Stock price analysis, portfolio P&L, risk metrics
Healthcare	Patient records, drug trial results, hospital KPIs
E-commerce	Sales trends, customer segmentation, inventory management
Marketing	Campaign performance, A/B test analysis, funnel analytics
Manufacturing	Quality control logs, sensor data aggregation

Installing Pandas

```
# Using pip
pip install pandas

# Using conda
conda install pandas

# Verify installation
import pandas as pd
print(pd.__version__)    # e.g., 2.2.0
```

Tip: Always import pandas as pd — this is the universal convention.

Pandas Data Structures Overview

Pandas has three primary data structures:

Structure	Dimensions	Description	Analogy
Series	1D	Labeled array of any dtype	Single column in Excel
DataFrame	2D	Tabular data — rows and columns	Entire Excel spreadsheet
Index	1D	Labels for rows or columns	Row numbers / header row

In Pandas 1.x there was also a Panel (3D) structure, but it was deprecated. For 3D+ data, use xarray or MultiIndex DataFrames.

10.2 Series

A Series is a one-dimensional labeled array capable of holding any data type (integers, floats, strings, Python objects, etc.). Think of it as a single column from a spreadsheet, with an index attached to each value.

Creating a Series

```
import pandas as pd
import numpy as np

# From a list
s1 = pd.Series([10, 20, 30, 40])
print(s1)
# 0    10
# 1    20   <- auto-generated integer index
# 2    30
# 3    40

# From a list with custom index
s2 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

# From a dictionary (keys become index)
s3 = pd.Series({'Jan': 100, 'Feb': 200, 'Mar': 150})

# From a NumPy array
s4 = pd.Series(np.array([1.1, 2.2, 3.3]), name='measurements')

# Scalar broadcast
s5 = pd.Series(5, index=['a','b','c','d'])  # All values = 5
```

Series Attributes

Attribute	Description	Example
s.index	The axis labels (Index object)	RangeIndex(start=0, stop=4)
s.values	Underlying NumPy array	array([10, 20, 30, 40])
s.dtype	Data type of elements	int64, float64, object
s.name	Name of the Series	'sales'
s.shape	Tuple of dimensions	(4,)
s.size	Total number of elements	4

Indexing and Slicing

```
s = pd.Series([10, 20, 30, 40, 50], index=['a','b','c','d','e'])

# Label-based access
s['a']          # 10
s[['a','c']]    # Multiple labels

# Position-based access
s.iloc[0]       # 10
s.iloc[1:3]     # b=20, c=30

# Boolean masking
```

```
s[s > 25]       # c=30, d=40, e=50
```

Vectorized Operations

```
s = pd.Series([10, 20, 30, 40])

# Arithmetic — element-wise
s + 5           # 15, 25, 35, 45
s * 2           # 20, 40, 60, 80
s ** 2          # 100, 400, 900, 1600

# Operations between two Series — aligns by index
s1 = pd.Series({'a':1, 'b':2})
s2 = pd.Series({'b':10, 'c':20})
s1 + s2         # a=NaN, b=12, c=NaN  (index alignment)

# Common methods
s.sum()         # 100
s.mean()        # 25.0
s.cumsum()      # Running total
s.describe()    # Count, mean, std, min, percentiles, max
```

Tip: Series alignment by index is one of the most powerful features of Pandas — it avoids many off-by-one errors common in raw Python/NumPy.

10.3 DataFrames

A DataFrame is a 2-dimensional labeled data structure with columns of potentially different types — essentially a table or spreadsheet in Python. It is the most commonly used Pandas object.

Creating DataFrames

```
import pandas as pd, numpy as np

# --- From a dictionary of lists ---
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', 'Charlie'],
    'Age':    [25, 30, 35],
    'Salary': [50000, 60000, 70000]
})

# --- From a list of dicts (each dict = one row) ---
records = [{'Name':'Alice','Score':90}, {'Name':'Bob','Score':85}]
df2 = pd.DataFrame(records)

# --- From a NumPy array ---
arr = np.random.rand(4, 3)
df3 = pd.DataFrame(arr, columns=['A','B','C'])

# --- From a CSV file ---
df4 = pd.read_csv('data.csv')   # Covered in detail in 10.5
```

Essential DataFrame Attributes

```
df.shape       # (rows, cols)   e.g., (3, 3)
df.columns     # Index(['Name', 'Age', 'Salary'])
df.dtypes      # dtype per column
df.index       # Row index
df.size        # Total elements = rows * cols
df.ndim        # Always 2 for DataFrame

df.info()      # Non-null count, dtypes, memory usage
df.describe()  # Statistical summary of numeric columns
df.head(5)     # First 5 rows (default)
df.tail(5)     # Last 5 rows
df.sample(3)   # 3 random rows
```

Selecting Columns

```
# Single column → Series
df['Name']          # or df.Name (dot notation, caution with spaces)

# Multiple columns → DataFrame
df[['Name', 'Salary']]

# Conditional column selection
num_cols = df.select_dtypes(include='number').columns
```

Selecting Rows

```
df.head(3)        # First 3 rows
df.tail(2)        # Last 2 rows
```

```
df.sample(5)      # 5 random rows
df.sample(frac=0.1)  # 10% sample
```

df.describe() only summarizes numeric columns by default. Use df.describe(include='all') to include object/categorical columns.

Tip: Use df.info() at the start of any analysis — it quickly reveals missing data and wrong data types.

10.4 Data Indexing and Selection

Pandas offers multiple ways to select data. The two primary accessors are .loc[] for label-based and .iloc[] for position-based indexing. Understanding the difference is critical for correct data selection.

.loc[] — Label-Based Selection

Use .loc[] when you know the index labels (row names) and/or column names.

```
import pandas as pd
df = pd.DataFrame({
    'Name':['Alice','Bob','Charlie','Dave'],
    'Age':[25,30,35,28],
    'Score':[90,85,92,88]
}, index=['r1','r2','r3','r4'])

# Single row
df.loc['r1']                    # Series for row r1

# Slice rows (inclusive on both ends with loc)
df.loc['r1':'r3']               # rows r1, r2, r3

# Row + column selection
df.loc['r1', 'Age']             # 25
df.loc['r1':'r2', ['Name','Score']]

# All rows, specific columns
df.loc[:, 'Age']                # Same as df['Age']
```

.iloc[] — Integer Position-Based Selection

Use .iloc[] when you want to select by position (0-based), ignoring the index labels.

```
# Single element
df.iloc[0, 1]        # Row 0, Col 1 → 25

# Rows slice (exclusive end like Python lists)
df.iloc[0:2]         # Rows 0 and 1 only

# Row + column slice
df.iloc[0:3, 1:3]    # Rows 0-2, Cols 1-2

# Last row
df.iloc[-1]

# Every other row
df.iloc[::2]
```

Feature	.loc[]	.iloc[]
Selection type	Label-based	Integer position-based
End of slice	Inclusive	Exclusive (like Python lists)
When to use	When index has meaningful labels	When you need positional access
Works with	Any index type	Always works (uses 0-based integers)

Boolean Indexing

```
# Single condition
df[df['Age'] > 28]

# Compound conditions
df[(df['Age'] > 28) & (df['Score'] >= 90)]   # AND
df[(df['Age'] < 26) | (df['Score'] > 91)]    # OR

# NOT condition
df[~(df['Age'] > 30)]

# isin() — match a list
df[df['Name'].isin(['Alice', 'Charlie'])]

# between()
df[df['Score'].between(85, 90)]
```

⚠ *Never chain boolean conditions with 'and'/'or' Python keywords — use & and | operators. Always wrap each condition in parentheses.*

Setting and Resetting Index

```
# Set a column as index
df = df.set_index('Name')      # Name column becomes row index

# Reset back to integer index
df = df.reset_index()          # Name moves back to column

# Set index in-place
df.set_index('Name', inplace=True)
```

Multi-Level (Hierarchical) Indexing

```
# Creating a MultiIndex DataFrame
arrays = [['Math','Math','Sci','Sci'], ['Q1','Q2','Q1','Q2']]
index = pd.MultiIndex.from_arrays(arrays, names=['Subject','Quarter'])
df_mi = pd.DataFrame({'Score':[85,90,78,88]}, index=index)

# Selecting with .loc on MultiIndex
df_mi.loc['Math']             # All Math rows
df_mi.loc[('Math','Q1')]      # Single cell

# Cross-section selection
df_mi.xs('Q1', level='Quarter')   # Q1 for all subjects
```

💡 ***Tip: MultiIndex is powerful for hierarchical data like time series with multiple instruments, or data grouped by region and category.***

10.5 Data Import / Export ★ Core Training Topic

Pandas provides a rich suite of I/O functions to read and write data in virtually every common format. Mastering these is essential for real-world data work.

10.5.1 Reading CSV — pd.read_csv()

CSV is the most common data exchange format. pd.read_csv() is highly configurable.

```
import pandas as pd

# Basic read
df = pd.read_csv('sales.csv')

# Key parameters explained:
df = pd.read_csv(
    'sales.csv',
    sep=',',              # Delimiter (use sep='\t' for TSV)
    header=0,             # Row number of column names (0=first row)
    # header=None,        # Use when file has NO header row
    names=['id','product','amount','date'],  # Custom column names
    dtype={'id': int, 'amount': float},      # Force column data types
    parse_dates=['date'],                    # Parse as datetime
    encoding='utf-8',     # File encoding (try 'latin-1' if utf-8 fails)
    nrows=1000,           # Read only first 1000 rows (good for preview)
    skiprows=2,           # Skip first 2 rows
    na_values=['N/A','--','?'],  # Treat these as NaN
    index_col='id',       # Use 'id' column as row index
)

# Chunked reading for large files
chunk_iter = pd.read_csv('big_file.csv', chunksize=50000)
results = []
for chunk in chunk_iter:
    results.append(chunk[chunk['amount'] > 100])
df_filtered = pd.concat(results, ignore_index=True)
```

Tip: For large CSV files (>1 GB), always use chunksize to avoid memory errors. Process each chunk and concatenate only what you need.

10.5.2 Reading Excel — pd.read_excel()

```
# Requires: pip install openpyxl

# Read first sheet
df = pd.read_excel('report.xlsx')

# Specify sheet by name or index
df = pd.read_excel('report.xlsx', sheet_name='Sales')
df = pd.read_excel('report.xlsx', sheet_name=0)    # First sheet by index

# Read ALL sheets → returns dict of {sheet_name: DataFrame}
all_sheets = pd.read_excel('report.xlsx', sheet_name=None)
df_sales   = all_sheets['Sales']
df_returns = all_sheets['Returns']

# Common parameters (same as read_csv)
df = pd.read_excel('report.xlsx',
    sheet_name='Data',
    header=1,             # Header is on row 2 (0-indexed)
```

```
    usecols='A:E',        # Read only columns A through E
    nrows=500,
    dtype={'Code': str},  # Prevent leading zeros from dropping
)
```

Engine	File Type	Install
openpyxl	.xlsx (Excel 2007+)	pip install openpyxl
xlrd	.xls (Legacy Excel 97-2003)	pip install xlrd
xlwt	Write .xls (legacy)	pip install xlwt

⚠ *xlrd no longer supports .xlsx files (since v2.0). Use openpyxl for .xlsx. Specify engine explicitly if needed: pd.read_excel(..., engine='openpyxl')*

10.5.3 Reading JSON — pd.read_json()

```
# Simple flat JSON array
df = pd.read_json('data.json')

# JSON string
json_str = '[{"name":"Alice","age":25},{"name":"Bob","age":30}]'
df = pd.read_json(json_str)

# Nested JSON — use json_normalize()
import json
from pandas import json_normalize

with open('nested.json') as f:
    data = json.load(f)

# Example nested structure:
# [{"id":1, "user":{"name":"Alice", "city":"Delhi"}}]
df = json_normalize(data,
    record_path=None,       # Path to nested list (if any)
    meta=['id'],            # Top-level fields to include
    sep='_'                 # Separator for nested keys → user_name, user_city
)
```

10.5.4 Reading SQL — pd.read_sql()

```
import sqlite3, sqlalchemy

# --- Using sqlite3 ---
conn = sqlite3.connect('mydb.sqlite')
df = pd.read_sql('SELECT * FROM orders WHERE amount > 100', conn)
conn.close()

# --- Using SQLAlchemy (recommended for production) ---
engine = sqlalchemy.create_engine('sqlite:///mydb.sqlite')
# For PostgreSQL: create_engine('postgresql://user:pass@host:5432/dbname')
# For MySQL:      create_engine('mysql+pymysql://user:pass@host/dbname')

# read_sql_query — for SQL strings
df = pd.read_sql_query('SELECT * FROM sales LIMIT 100', engine)

# read_sql_table — for entire tables
df = pd.read_sql_table('sales', engine)
```

```
# Parameterized query (prevents SQL injection)
params = {'min_amount': 500}
df = pd.read_sql('SELECT * FROM orders WHERE amount > :min_amount', engine,
params=params)

# With chunked reading
for chunk in pd.read_sql('SELECT * FROM big_table', engine, chunksize=10000):
    process(chunk)
```

⚠ *Never use string formatting to build SQL queries (e.g., f'WHERE id={user_input}'). Always use parameterized queries to prevent SQL injection.*

10.5.5 Reading HTML Tables — pd.read_html()

```
# Requires: pip install lxml (or html5lib)

# Returns a LIST of DataFrames (one per <table> found on page)
tables = pd.read_html('https://en.wikipedia.org/wiki/List_of_countries_by_GDP')
print(f'Found {len(tables)} tables')
df = tables[0]   # First table

# From local HTML file
tables = pd.read_html('local_report.html')

# Filter by table attributes
tables = pd.read_html(url, attrs={'id': 'gdp-table'})

# Handle thousands separator and decimal
df = pd.read_html(url, thousands=',', decimal='.')[0]
```

10.5.6 Other Formats

```
# Parquet — columnar format, great for large datasets
# pip install pyarrow
df = pd.read_parquet('data.parquet')
df = pd.read_parquet('data.parquet', columns=['id','amount'])  # Partial read

# Feather — fast binary format
# pip install pyarrow
df = pd.read_feather('data.feather')

# Clipboard (reads from system clipboard)
df = pd.read_clipboard()   # After copying a table from Excel/browser
```

10.5.7 Writing / Exporting Data

```
# --- CSV ---
df.to_csv('output.csv', index=False)       # index=False: don't write row numbers
df.to_csv('output.csv.gz', compression='gzip')  # Compressed
df.to_csv('output.csv', encoding='utf-8-sig')    # BOM for Excel compatibility

# --- Excel ---
df.to_excel('output.xlsx', index=False, sheet_name='Results')

# Multiple sheets
with pd.ExcelWriter('multi.xlsx', engine='openpyxl') as writer:
```

```
    df_sales.to_excel(writer, sheet_name='Sales', index=False)
    df_returns.to_excel(writer, sheet_name='Returns', index=False)

# --- JSON ---
df.to_json('output.json', orient='records', indent=2)
# orient options: 'records', 'columns', 'index', 'split', 'table'

# --- SQL ---
df.to_sql('new_table', engine, if_exists='replace', index=False)
# if_exists: 'fail' | 'replace' | 'append'

# --- Parquet ---
df.to_parquet('output.parquet', index=False, compression='snappy')
```

Format	Best For	Advantage
CSV	Data exchange, Excel users	Universal, human-readable
Excel	Business reports, stakeholders	Supports formatting, multiple sheets
JSON	APIs, web applications	Nested structures, web-compatible
SQL	Database storage, queries	Queryable, relational integrity
Parquet	Big data, analytics	Columnar, compressed, very fast

10.6 Data Cleaning

Raw data is almost never clean. Pandas provides comprehensive tools to detect, handle, and fix data quality issues. Good data cleaning is the foundation of reliable analysis. In this section, we use a real-world sales dataset (sales.csv) containing daily transaction records for electronics products across regions.

Sample Dataset — sales.csv

The dataset has five columns: date, product, quantity, price, and region. It intentionally contains missing values in quantity and region to demonstrate cleaning techniques.

#	date	product	quantity	price	region
1	2024-01-01	Monitor	13	18000	—
2	2024-01-02	Headphones	NaN	18000	East
3	2024-01-03	Tablet	8	15000	East
4	2024-01-04	Headphones	NaN	15000	South
5	2024-01-05	Monitor	5	15000	South
6	2024-01-06	Headphones	12	75000	East
7	2024-01-07	Tablet	NaN	15000	West
8	2024-01-08	Tablet	12	15000	North

#	date	product	quantity	price	region
...	...	...	...	...	...
17	2024-01-17	Monitor	NaN	3000	NaN
30	2024-01-30	Laptop	4	15000	NaN

Rows 17 and 30 have missing values in both quantity and region — visible even in a quick preview. NaN values are highlighted in red above.

Detecting Missing Data

Before fixing missing data, always measure how much is missing and where. Pandas provides several methods to detect NaN values across the sales DataFrame.

```
import pandas as pd
import numpy as np

df = pd.read_csv('sales.csv', parse_dates=['date'])

# Check for missing values — returns True where NaN
df.isnull()
df.isna()              # Same as isnull()
df.notnull()           # True where NOT NaN

# Count missing per column
df.isnull().sum()
# date        0
# product     0
# quantity    13    <-- 13 rows have no quantity
# price        0
# region       2    <-- 2 rows have no region

# Percentage missing per column
(df.isnull().sum() / len(df) * 100).round(2)
# quantity    43.33
# region       6.67

# Any missing in the entire DataFrame?
df.isnull().any().any()    # True

# Heatmap visualization (with seaborn)
# import seaborn as sns
# sns.heatmap(df.isnull(), cbar=False, yticklabels=False)
```

Tip: Check missingness patterns early. Here, quantity is missing in 43% of rows — too many to simply drop. Region is only missing in 2 rows, making it safe to drop or fill with a default.

Handling Missing Data

The right strategy depends on why data is missing. In our sales dataset, missing quantity may mean the sale was not completed, while missing region is likely a data entry error.

```
# --- dropna() ---
df.dropna()                          # Drops all 13 rows with ANY missing value
df.dropna(axis=1)                    # Drop entire 'quantity' and 'region' columns
df.dropna(thresh=4)                  # Keep rows with at least 4 non-NaN values
df.dropna(subset=['region'])         # Drop only the 2 rows missing region

# --- fillna() ---
df.fillna(0)                                    # Fill all NaN with 0
df['quantity'].fillna(df['quantity'].mean())    # Fill with mean qty (≈ 8.8)
df['quantity'].fillna(df['quantity'].median()) # Fill with median qty (8.0)
df.fillna(method='ffill')                       # Forward fill (use previous row's
value)
df.fillna(method='bfill')                       # Backward fill

# Fill each column differently
df.fillna({'quantity': df['quantity'].median(), 'region': 'Unknown'})

# --- interpolate() ---
# Useful for quantity since dates are sequential
df['quantity'].interpolate()                    # Linear interpolation
df['quantity'].interpolate(method='time')       # Time-aware interpolation
```

Tip: For quantity, interpolation works well since dates are sequential — it estimates missing sales from surrounding days. For region, use fillna('Unknown') or drop those 2 rows since region is categorical and can't be interpolated.

Removing Duplicates

Duplicate rows can silently inflate totals. In a sales dataset, a duplicate row could mean double-counting a transaction. Always check before aggregating.

```
# Detect duplicates
df.duplicated()                              # True for fully duplicate rows
df.duplicated(subset=['date', 'product'])    # Same product sold twice on same
date

# Count duplicates
df.duplicated().sum()                        # How many duplicate rows exist?

# Remove duplicates
df.drop_duplicates()                         # Keep first occurrence
df.drop_duplicates(keep='last')              # Keep last occurrence
df.drop_duplicates(keep=False)               # Drop ALL copies of any duplicate
df.drop_duplicates(subset=['date','product'])# Deduplicate by date + product
```

Data Type Conversion

After loading sales.csv, check that each column has the right dtype. Pandas may read quantity as float64 due to NaN values, and date as object (string) if not parsed on load.

```
# View current dtypes
df.dtypes
# date        object   (if not parsed on load)
```

```
# product     object
# quantity    float64  (NaN forces float)
# price       int64
# region      object

# Convert date column to datetime
df['date'] = pd.to_datetime(df['date'])                    # Auto-detect format
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d') # Explicit format
df['date'] = pd.to_datetime(df['date'], errors='coerce')   # Bad dates → NaT

# After filling NaN, convert quantity to int
df['quantity'] = df['quantity'].fillna(0).astype(int)

# Safer conversion with error handling
df['price'] = pd.to_numeric(df['price'], errors='coerce')  # Non-numeric → NaN

# Convert product and region to category (saves memory)
df['product'] = df['product'].astype('category')
df['region']  = df['region'].astype('category')
```

Renaming Columns and Replacing Values

Standardize column names and replace coded or inconsistent values before analysis.

```
# Rename specific columns
df.rename(columns={'price': 'unit_price', 'quantity': 'qty'}, inplace=True)

# Rename all columns at once
df.columns = ['date', 'product', 'qty', 'unit_price', 'region']

# Clean column names — lowercase and replace spaces
df.columns = df.columns.str.lower().str.replace(' ', '_')

# Standardize region names (fix typos or abbreviations)
df['region'].replace({'N': 'North', 'S': 'South', 'E': 'East', 'W': 'West'},
inplace=True)

# Replace empty strings with NaN
df.replace({'': np.nan}, inplace=True)

# Normalize product names to title case
df['product'] = df['product'].str.strip().str.title()
```

10.7 Data Transformation

Transformation reshapes, enriches, or modifies data while preserving or enhancing its meaning. With a clean sales dataset, we can now sort, filter, add computed columns, and apply custom logic using Pandas' rich toolkit.

Sorting Data

Sort sales data by date, quantity, or price to identify trends, top sellers, or outliers.

```
# Sort by a single column
df.sort_values('quantity')                                  # Ascending (lowest qty
first)
df.sort_values('price', ascending=False)                   # Descending (most
expensive first)

# Multi-column sort: region A→Z, then price high→low within each region
df.sort_values(['region', 'price'], ascending=[True, False])

# Sort by date (chronological)
df.sort_values('date', ascending=True)

# Sort by index
df.sort_index()                 # Reset to original row order
df.sort_index(ascending=False) # Reverse row order
```

Filtering Rows

Filter the sales data to focus on specific products, regions, or value ranges for targeted analysis.

```
# Filter by condition
df_high_value = df[df['price'] >= 18000]          # Sales ≥ ₹18,000
df_north      = df[df['region'] == 'North']       # North region only

# Multiple conditions
df[(df['product'] == 'Laptop') & (df['region'] == 'East')]

# Filter using query() — readable string syntax
df.query('price > 15000 and region == "North"')
df.query('quantity.between(5, 12)')               # qty between 5 and 12

# Filter with string methods
df[df['product'].str.startswith('H')]             # Headphones
df[df['region'].str.contains('outh')]             # South

# Filter rows where quantity is missing
df[df['quantity'].isna()]                         # 13 rows with no qty
```

Adding and Removing Columns

Enrich the sales dataset with computed columns like revenue, tax, and performance categories.

```
# Compute revenue per row
df['revenue'] = df['quantity'] * df['price']

# Apply 18% GST to get total amount
df['total_with_gst'] = df['revenue'] * 1.18

# Conditional column using np.where
import numpy as np
df['is_high_value'] = np.where(df['price'] >= 18000, 'High', 'Standard')

# Multiple conditions using np.select
conditions = [df['price'] >= 75000, df['price'] >= 18000, df['price'] >= 15000]
choices    = ['Premium',            'High',               'Mid-range']
df['price_tier'] = np.select(conditions, choices, default='Budget')
```

```
# Monitor at ₹18,000 → 'High'
# Headphones at ₹75,000 → 'Premium'
# Tablet at ₹3,000 → 'Budget'

# Remove helper column
df.drop(columns=['is_high_value'], inplace=True)
df.drop(columns=['total_with_gst', 'price_tier'])  # Multiple columns
```

apply() — Row-wise and Column-wise Functions

Use apply() to run custom functions across columns or rows of the sales DataFrame.

```
# Column-wise (axis=0, default): range of values per numeric column
df[['quantity','price']].apply(lambda col: col.max() - col.min())
# quantity     10.0
# price     72000

# Row-wise (axis=1): combine fields per row
df['label'] = df.apply(
    lambda row: f"{row['product']} ({row['region']})", axis=1
)
# → 'Monitor (North)', 'Headphones (East)', ...

# Using a named function for price tier
def price_tier(price):
    if price >= 75000: return 'Premium'
    elif price >= 18000: return 'High'
    elif price >= 15000: return 'Mid-range'
    else: return 'Budget'

df['price_tier'] = df['price'].apply(price_tier)
```

map() — Element-wise Mapping on Series

Use map() to transform individual values in a Series using a dictionary or function — ideal for encoding categorical columns.

```
# Map region to a numeric zone code
df['zone_code'] = df['region'].map({'North': 1, 'South': 2, 'East': 3, 'West': 4})

# Map product to a category ID
product_ids = {'Monitor': 101, 'Headphones': 102, 'Tablet': 103,
               'Phone': 104, 'Laptop': 105}
df['product_id'] = df['product'].map(product_ids)

# Map using a function — normalize price to 0-1 scale
max_price = df['price'].max()   # 75000
df['price_norm'] = df['price'].map(lambda x: round(x / max_price, 4))
# ₹75,000 → 1.0000 | ₹3,000 → 0.0400
```

DataFrame.map() — Element-wise on Entire DataFrame

Apply a function to every individual cell across multiple columns at once — useful for formatting or capping values across numeric columns.

```
# Select only numeric columns from the sales data
df_nums = df[['quantity', 'price']].copy()

# Pandas 2.1+: DataFrame.map() (replaces deprecated applymap)
df_nums.map(lambda x: round(x, 2) if pd.notna(x) else x)

# Cap quantity at 10 across all numeric columns
df_nums.map(lambda x: min(x, 10) if pd.notna(x) else x)

# Pandas < 2.1 — use applymap() (now deprecated)
df_nums.applymap(lambda x: round(x, 2))
```

⚠ **Warning:** applymap() was deprecated in Pandas 2.1 and renamed to DataFrame.map(). Always handle NaN values explicitly inside the lambda (e.g., check pd.notna(x)) since the sales quantity column contains missing values.

10.6 Data Cleaning

Raw data is almost never clean. Pandas provides comprehensive tools to detect, handle, and fix data quality issues. Good data cleaning is the foundation of reliable analysis.

Detecting Missing Data

```
import pandas as pd, numpy as np
df = pd.DataFrame({'A':[1,np.nan,3], 'B':[np.nan,2,np.nan], 'C':[1,2,3]})

# Check for missing values
df.isnull()          # Boolean DataFrame — True where NaN
df.isna()            # Same as isnull()
df.notnull()         # Opposite — True where NOT NaN

# Count missing per column
df.isnull().sum()

# Percentage missing per column
(df.isnull().sum() / len(df) * 100).round(2)

# Any missing in entire DataFrame?
df.isnull().any().any()    # True/False

# Heatmap visualization (with seaborn)
# import seaborn as sns
# sns.heatmap(df.isnull(), cbar=False)
```

Handling Missing Data

```
# --- dropna() ---
df.dropna()                      # Drop rows with ANY missing value
df.dropna(axis=1)                # Drop COLUMNS with any missing value
df.dropna(thresh=2)              # Keep rows with at least 2 non-NaN values
df.dropna(subset=['A','B'])      # Drop rows where A or B is missing

# --- fillna() ---
df.fillna(0)                     # Fill all NaN with 0
df['A'].fillna(df['A'].mean())   # Fill with column mean
df.fillna(method='ffill')        # Forward fill (propagate last valid value)
df.fillna(method='bfill')        # Backward fill
df.fillna({'A': 0, 'B': 'Unknown'})   # Fill each column differently

# --- interpolate() ---
df['A'].interpolate()            # Linear interpolation between values
df['A'].interpolate(method='time') # Time-aware interpolation
```

Tip: For numeric data, consider the nature of missingness before filling: if missing means 0 (e.g., no sales), use fillna(0). If it is truly unknown, use mean/median or interpolation.

Removing Duplicates

```
# Detect duplicates
df.duplicated()                      # Boolean Series — True for duplicate rows
df.duplicated(subset=['Name','Date'])  # Check specific columns only

# Count duplicates
df.duplicated().sum()
```

```
# Remove duplicates
df.drop_duplicates()                    # Keep first occurrence
df.drop_duplicates(keep='last')         # Keep last occurrence
df.drop_duplicates(keep=False)          # Drop ALL duplicate rows
df.drop_duplicates(subset=['id'])       # Based on 'id' column only
```

Data Type Conversion

```
# View current dtypes
df.dtypes

# astype() — explicit conversion
df['age']    = df['age'].astype(int)
df['score']  = df['score'].astype(float)
df['name']   = df['name'].astype(str)

# pd.to_numeric() — safer (handles errors)
df['amount'] = pd.to_numeric(df['amount'], errors='coerce')  # Non-numeric → NaN

# pd.to_datetime() — parse date strings
df['date'] = pd.to_datetime(df['date'])                      # Auto detect format
df['date'] = pd.to_datetime(df['date'], format='%d/%m/%Y')   # Explicit format
df['date'] = pd.to_datetime(df['date'], errors='coerce')     # Bad dates → NaT
```

Renaming Columns and Replacing Values

```
# Rename specific columns
df.rename(columns={'old_name': 'new_name', 'qty': 'quantity'}, inplace=True)

# Rename all columns at once (list must match length)
df.columns = ['id', 'product', 'amount', 'date']

# Clean column names (lowercase, replace spaces)
df.columns = df.columns.str.lower().str.replace(' ', '_')

# Replace specific values
df['status'].replace({'Y': True, 'N': False}, inplace=True)
df.replace({'': np.nan}, inplace=True)   # Replace empty strings with NaN
```

10.7 Data Transformation

Transformation involves reshaping, enriching, or modifying data while preserving or enhancing its meaning. Pandas offers a rich set of tools for these operations.

Sorting Data

```
# Sort by values
df.sort_values('Age')                              # Ascending (default)
df.sort_values('Age', ascending=False)             # Descending
df.sort_values(['City','Age'], ascending=[True,False])  # Multi-column sort

# Sort by index
df.sort_index()                                    # Ascending
df.sort_index(ascending=False)
```

Filtering Rows

```
# Condition-based filter
df_seniors = df[df['Age'] >= 60]

# Filter with query() — concise string syntax
df.query('Age > 30 and City == "Delhi"')
df.query('Score.between(80, 95)')                  # Using method

# Filter with string conditions
df[df['Name'].str.startswith('A')]
df[df['Email'].str.contains('@gmail')]
```

Adding and Removing Columns

```
# Add new column
df['Tax']        = df['Salary'] * 0.20
df['Full_Name']  = df['First'] + ' ' + df['Last']

# Conditional column (np.where)
import numpy as np
df['Category'] = np.where(df['Score'] >= 75, 'Pass', 'Fail')

# Multiple conditions (np.select)
conditions = [df['Score']>=90, df['Score']>=75, df['Score']>=60]
choices    = ['A',             'B',             'C']
df['Grade'] = np.select(conditions, choices, default='F')

# Remove column
df.drop(columns=['Tax'], inplace=True)
df.drop(columns=['Col1','Col2'])     # Multiple
```

apply() — Row-wise and Column-wise Functions

```
# Column-wise (axis=0, default): applies function to each column
df.apply(lambda col: col.max() - col.min())     # Range per column

# Row-wise (axis=1): applies function to each row
df['Total'] = df.apply(lambda row: row['Math'] + row['Science'], axis=1)
```

```
# Using a named function
def categorize(val):
    return 'High' if val > 70 else 'Low'

df['Category'] = df['Score'].apply(categorize)
```

map() — Element-wise Mapping on Series

```
# Map values using a dictionary
df['Gender_Code'] = df['Gender'].map({'Male': 1, 'Female': 0})

# Map using a function
df['Score_Norm'] = df['Score'].map(lambda x: round(x / 100, 2))
```

applymap() / DataFrame.map() — Element-wise on DataFrame

```
# Pandas < 2.1: applymap()
df_nums = df.select_dtypes(include='number')
df_nums.applymap(lambda x: round(x, 2))

# Pandas 2.1+: DataFrame.map() replaces applymap()
df_nums.map(lambda x: round(x, 2))
```

⚠ *applymap() was deprecated in Pandas 2.1 and renamed to DataFrame.map(). If you see warnings, update your code to use .map() on the DataFrame.*

10.8 Grouping and Aggregation

groupby() implements the split-apply-combine paradigm: split the DataFrame into groups, apply a function to each group, and combine the results. It is one of the most powerful features of Pandas.

Basic groupby()

```
import pandas as pd
df = pd.DataFrame({
    'Region':  ['North','South','North','South','East'],
    'Product': ['A','B','A','A','B'],
    'Sales':   [100, 200, 150, 300, 250],
    'Units':   [10,  20,  15,  30,  25]
})

# Single key groupby
df.groupby('Region')['Sales'].sum()

# Multiple keys
df.groupby(['Region','Product'])['Sales'].sum()

# Built-in aggregations
df.groupby('Region')['Sales'].mean()
df.groupby('Region')['Sales'].count()
df.groupby('Region')['Sales'].min()
df.groupby('Region')['Sales'].max()
df.groupby('Region')['Sales'].nunique()
```

Multiple Aggregations with agg()

```
# Multiple functions on one column
df.groupby('Region')['Sales'].agg(['sum','mean','count','std'])

# Different functions on different columns
df.groupby('Region').agg({
    'Sales': ['sum','mean'],
    'Units': 'sum'
})

# Named aggregations (Pandas 0.25+) — clean output column names
df.groupby('Region').agg(
    Total_Sales = ('Sales', 'sum'),
    Avg_Sales   = ('Sales', 'mean'),
    Total_Units = ('Units', 'sum')
)
```

Pivot Tables — pd.pivot_table()

```
# Basic pivot table
pivot = pd.pivot_table(df,
    values='Sales',         # Values to aggregate
    index='Region',         # Row grouping
    columns='Product',      # Column grouping
    aggfunc='sum',          # Aggregation function
    fill_value=0            # Replace NaN with 0
)

# Multiple value columns
pivot2 = pd.pivot_table(df,
```

```
    values=['Sales','Units'],
    index='Region',
    columns='Product',
    aggfunc='sum',
    margins=True,           # Add grand totals (row/column)
    margins_name='Total'
)
```

Cross-Tabulations — pd.crosstab()

```
# Frequency table
pd.crosstab(df['Region'], df['Product'])

# With percentages (normalize)
pd.crosstab(df['Region'], df['Product'], normalize='index')  # Row %
pd.crosstab(df['Region'], df['Product'], normalize='columns') # Column %
```

transform() — Group-Aware Transformation

Unlike agg(), transform() returns a Series/DataFrame of the same shape as the input — useful for adding group-level statistics back to the original DataFrame.

```
# Add group mean as a new column (without collapsing rows)
df['Region_Avg_Sales'] = df.groupby('Region')['Sales'].transform('mean')

# Normalize sales within each region
df['Sales_Normalized'] = df.groupby('Region')['Sales'].transform(
    lambda x: (x - x.mean()) / x.std()
)
```

filter() — Filter Groups

```
# Keep only groups where total sales > 300
df.groupby('Region').filter(lambda g: g['Sales'].sum() > 300)
```

10.9 Merging and Joining

Combining data from multiple sources is a daily task. Pandas provides merge(), concat(), and join() for different combination scenarios — all inspired by SQL JOIN operations.

merge() — SQL-Style Joins

```
import pandas as pd
orders = pd.DataFrame({'order_id':[1,2,3], 'customer_id':[101,102,103],
'amount':[500,300,700]})
customers = pd.DataFrame({'customer_id':[101,102,104],
'name':['Alice','Bob','Charlie']})

# INNER JOIN — only matching keys
pd.merge(orders, customers, on='customer_id', how='inner')

# LEFT JOIN — all rows from left, matching from right
pd.merge(orders, customers, on='customer_id', how='left')

# RIGHT JOIN — all rows from right, matching from left
pd.merge(orders, customers, on='customer_id', how='right')

# OUTER JOIN — all rows from both
pd.merge(orders, customers, on='customer_id', how='outer')

# Merge on different key names
pd.merge(orders, customers, left_on='customer_id', right_on='cust_id')

# Merge on multiple keys
pd.merge(df1, df2, on=['date','product_id'])

# Handling duplicate column names
pd.merge(df1, df2, on='id', suffixes=('_left','_right'))
```

Join Type	Result	SQL Equivalent
inner (default)	Only rows with matching keys in BOTH DataFrames	INNER JOIN
left	All rows from left, NaN where no match in right	LEFT JOIN
right	All rows from right, NaN where no match in left	RIGHT JOIN
outer	All rows from both, NaN where no match	FULL OUTER JOIN

pd.concat() — Stacking DataFrames

```
# Row-wise concatenation (stack vertically)
combined = pd.concat([df_jan, df_feb, df_mar], ignore_index=True)

# Column-wise concatenation (stack horizontally)
combined = pd.concat([df_info, df_scores], axis=1)

# Keys to trace origin
combined = pd.concat([df1, df2], keys=['Source1','Source2'])
```

join() — Index-Based Joining

```
# Join on index (DataFrames must have matching index)
df1.join(df2, how='left')          # Left join on index

# Join on a column from left + index from right
df1.join(df2.set_index('key'), on='key')
```

Handling Key Mismatches and Duplicates

```
# Check for unexpected duplicates after merge
merged = pd.merge(df1, df2, on='id', how='inner')
assert len(merged) == expected_rows, 'Unexpected row count after merge!'

# Identify duplicate keys before merging
print('df1 id duplicates:', df1['id'].duplicated().sum())
print('df2 id duplicates:', df2['id'].duplicated().sum())
```

⚠ *Many-to-many merges can cause row explosion. Always verify row counts after a merge. If df1 has 100 rows and df2 has 100 rows, a merge can produce up to 10,000 rows if keys aren't unique.*

10.10 Time Series

Pandas has first-class support for time series data. It provides powerful tools for date/time indexing, resampling, rolling calculations, and time zone handling.

Parsing and Creating DatetimeIndex

```
import pandas as pd

# Create a DatetimeIndex
dates = pd.date_range('2024-01-01', periods=10, freq='D')   # Daily
dates = pd.date_range('2024-01-01', '2024-12-31', freq='M') # Monthly end
dates = pd.date_range('2024-01-01', periods=24, freq='H')   # Hourly

# Parse dates when reading
df = pd.read_csv('sales.csv', parse_dates=['date'], index_col='date')

# Convert a column manually
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date')    # Set as DatetimeIndex

# Access date components
df.index.year
df.index.month
df.index.day
df.index.dayofweek    # 0=Monday, 6=Sunday
df.index.quarter

# Filter by date range
df['2024']                # All 2024 data
df['2024-01':'2024-06']   # Jan-Jun 2024
```

Resampling — resample()

Resampling changes the frequency of time series data — aggregating or upsampling.

```
# Downsample — from daily to weekly sum
df.resample('W').sum()

# Monthly mean
df.resample('M').mean()

# Quarterly sum
df.resample('Q').sum()

# Hourly to 4-hourly
df.resample('4H').mean()
```

Alias	Frequency
D	Calendar day
W	Weekly
M	Month end
MS	Month start
Q	Quarter end
Y	Year end

H	Hour
T / min	Minute

Rolling and Expanding Windows

```
# 7-day rolling average
df['MA_7'] = df['Sales'].rolling(window=7).mean()

# Rolling with minimum periods
df['MA_7'] = df['Sales'].rolling(window=7, min_periods=1).mean()

# Expanding window — from start to current row
df['Cumulative_Max'] = df['Sales'].expanding().max()
df['Running_Mean']   = df['Sales'].expanding().mean()
```

Shifting, Lagging, and Differencing

```
# Shift values forward (create lag features)
df['Sales_Yesterday'] = df['Sales'].shift(1)    # 1-period lag
df['Sales_Last_Week'] = df['Sales'].shift(7)

# Shift backward
df['Sales_Tomorrow']  = df['Sales'].shift(-1)

# Day-over-day change
df['Change']     = df['Sales'].diff(1)

# Percentage change
df['Pct_Change'] = df['Sales'].pct_change()
```

Time Zone Handling

```
# Localize naive datetime to a timezone
df.index = df.index.tz_localize('UTC')
df.index = df.index.tz_localize('Asia/Kolkata')

# Convert between timezones
df.index = df.index.tz_convert('US/Eastern')

# Get current time with timezone
pd.Timestamp.now(tz='Asia/Kolkata')
```

10.11 Advanced Topics

Categorical Data Type

Converting string columns with limited unique values to categorical dtype saves memory and enables ordering.

```
import pandas as pd

# Convert to categorical
df['Grade'] = df['Grade'].astype('category')

# Memory comparison
print(df['Grade'].memory_usage(deep=True))   # After conversion — much smaller

# Ordered categories (for sorting/comparison)
from pandas.api.types import CategoricalDtype
grade_order = CategoricalDtype(['F','D','C','B','A'], ordered=True)
df['Grade'] = df['Grade'].astype(grade_order)

# Now comparisons work correctly
df[df['Grade'] >= 'B']    # B and A grades only
df.sort_values('Grade')   # Sorted F→D→C→B→A
```

Tip: Use categorical dtype for columns like 'Status', 'Region', 'Category', 'Gender' — it can reduce memory by 5-10x for large DataFrames.

String Operations via .str Accessor

```
s = pd.Series(['  Alice Smith ', 'BOB JONES', 'charlie brown'])

# Case operations
s.str.lower()          # 'alice smith', 'bob jones', ...
s.str.upper()
s.str.title()
s.str.strip()          # Remove leading/trailing whitespace

# Contains, starts, ends
s.str.contains('Alice')
s.str.startswith('B')
s.str.endswith('es')

# Split and extract
s.str.split()                    # Split on whitespace → list
s.str.split(expand=True)         # Split into separate columns
s.str.get(0)                     # First element after split

# Replace and extract with regex
s.str.replace(r'[^a-zA-Z ]', '', regex=True)
s.str.extract(r'(\w+)\s(\w+)')   # Extract first and last name

# String length and count
s.str.len()
s.str.count('s')   # Count occurrences of 's'
```

Exponential Weighted Moving Average — ewm()

```
# Exponential moving average (recent values weighted more)
df['EMA_10'] = df['Price'].ewm(span=10).mean()
```

```
# Compare to simple rolling average
df['SMA_10'] = df['Price'].rolling(10).mean()
# EMA reacts faster to price changes than SMA
```

10.12 Lab: End-to-End Data Wrangling

This lab exercise integrates all topics from Module 10 into a single realistic workflow. You will import data from multiple sources, clean and merge it, perform aggregations, and export the results.

Scenario

You work as a data analyst at a retail company. You have been given three data sources:

- sales.csv — daily transaction records (transaction_id, date, product_id, quantity, unit_price, region)
- products.xlsx — product master data (product_id, name, category, cost_price)
- customers table in a SQLite database — customer info (transaction_id, customer_id, customer_name, city)

Your task: Build a consolidated sales report and export it.

Step 1: Import Data from Multiple Sources

```
import pandas as pd, sqlite3

# Import CSV
sales = pd.read_csv('sales.csv', parse_dates=['date'],
                    dtype={'product_id': str})

# Import Excel
products = pd.read_excel('products.xlsx', sheet_name='Products',
                         dtype={'product_id': str})

# Import from SQLite
conn = sqlite3.connect('company.db')
customers = pd.read_sql('SELECT * FROM customers', conn)
conn.close()

print(sales.shape, products.shape, customers.shape)
```

Step 2: Inspect and Clean Data

```
# Inspect
sales.info()
sales.describe()
(sales.isnull().sum() / len(sales) * 100).round(2)

# Clean sales
sales.drop_duplicates(subset=['transaction_id'], inplace=True)
sales['quantity']   = pd.to_numeric(sales['quantity'],   errors='coerce')
sales['unit_price'] = pd.to_numeric(sales['unit_price'], errors='coerce')
sales.dropna(subset=['quantity','unit_price'], inplace=True)

# Derived column
sales['revenue'] = sales['quantity'] * sales['unit_price']

# Clean column names
products.columns = products.columns.str.lower().str.strip()
```

Step 3: Merge DataFrames

```
```

```
# Merge sales with products
df = pd.merge(sales, products[['product_id','name','category','cost_price']],
              on='product_id', how='left')

# Merge with customers
df = pd.merge(df, customers[['transaction_id','customer_name','city']],
              on='transaction_id', how='left')

# Add profit column
df['profit'] = df['revenue'] - (df['quantity'] * df['cost_price'])

print(f'Final shape: {df.shape}')
```

Step 4: Aggregate and Summarize

```
# Monthly revenue by category
df['month'] = df['date'].dt.to_period('M')
monthly_report = df.groupby(['month','category']).agg(
    Total_Revenue = ('revenue', 'sum'),
    Total_Profit  = ('profit',  'sum'),
    Transactions  = ('transaction_id', 'count'),
    Avg_Order     = ('revenue', 'mean')
).round(2)

# Pivot: category vs month
pivot = pd.pivot_table(df, values='revenue',
    index='category', columns='month',
    aggfunc='sum', fill_value=0).round(2)

# Top 10 customers
top_customers = df.groupby('customer_name')['revenue'].sum()\
    .sort_values(ascending=False).head(10)
```

Step 5: Export Results in Multiple Formats

```
# Export to CSV (lightweight, universal)
monthly_report.to_csv('monthly_report.csv')

# Export to Excel with multiple sheets
with pd.ExcelWriter('sales_analysis.xlsx', engine='openpyxl') as writer:
    monthly_report.to_excel(writer, sheet_name='Monthly_Revenue')
    pivot.to_excel(writer, sheet_name='Pivot_Category_Month')
    top_customers.to_excel(writer, sheet_name='Top_Customers')

# Export cleaned dataset as Parquet
df.to_parquet('cleaned_sales.parquet', index=False, compression='snappy')

print('All exports complete!')
```

Lab Success Criteria

- All three data sources imported without errors
- Missing values identified and handled appropriately
- Merged DataFrame has no unexpected duplicate rows
- Monthly revenue report correctly aggregated
- Exports created in CSV, Excel, and Parquet formats

💡 *Tip: Real-world data is messy. Always add assertions (assert len(df) == expected) and logging at each step to catch problems early.*

Quick Reference — Pandas Cheat Sheet

Import / Export

Operation	Code
Read CSV	pd.read_csv('file.csv', sep=',', parse_dates=['date'])
Read Excel	pd.read_excel('file.xlsx', sheet_name='Sheet1')
Read JSON	pd.read_json('file.json') / json_normalize(data)
Read SQL	pd.read_sql('SELECT ...', engine)
Write CSV	df.to_csv('out.csv', index=False)
Write Excel	df.to_excel('out.xlsx', index=False)
Write Parquet	df.to_parquet('out.parquet', compression='snappy')

Selection

Task	Code
Column	df['col'] or df[['col1','col2']]
Row by label	df.loc['label'] or df.loc['a':'c']
Row by position	df.iloc[0] or df.iloc[0:5]
Boolean filter	df[df['col'] > 10]
Compound filter	df[(df['A']>5) & (df['B']<10)]
Query	df.query('A > 5 and B < 10')

Cleaning

Task	Code
Detect missing	df.isnull().sum()
Drop missing rows	df.dropna() or df.dropna(subset=['col'])
Fill missing	df.fillna(0) or df['col'].fillna(df['col'].mean())
Remove duplicates	df.drop_duplicates(subset=['id'])
Convert dtype	df['col'].astype(int) or pd.to_datetime(df['col'])
Rename columns	df.rename(columns={'old':'new'})

Grouping

Task	Code
Group and sum	df.groupby('col')['val'].sum()
Multiple aggs	df.groupby('col').agg({'A':'sum', 'B':'mean'})
Named agg	df.groupby('col').agg(Total=('val','sum'))
Pivot table	pd.pivot_table(df, values='v', index='r', columns='c', aggfunc='sum')
Cross-tab	pd.crosstab(df['A'], df['B'])

Transform	df.groupby('col')['val'].transform('mean')

Time Series

Task	Code
Parse dates	pd.to_datetime(df['date'])
Date range	pd.date_range('2024-01-01', periods=12, freq='M')
Resample monthly	df.resample('M').sum()
Rolling average	df['col'].rolling(7).mean()
Lag feature	df['col'].shift(1)
Period change	df['col'].diff() / df['col'].pct_change()

Vikas Karanth U at 2/21/2026 10:47 PM